ADI Internship

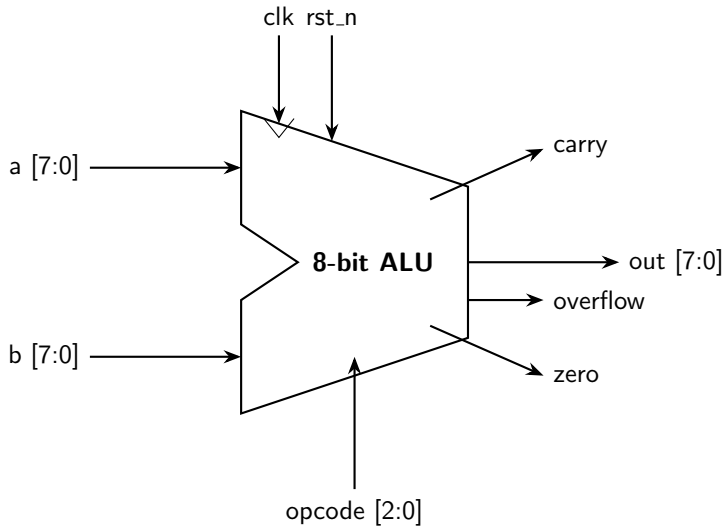
ALU Verification Plan

Name: Yossef Medhat Ali

Submitted to: Eng. Mohamed Ahmed El-adawy

July 24, 2026

ALU diagram and specification:



Block Diagram of a Synchronous 8-bit ALU

- 8-bit Arithmetic Logic Unit (ALU).
- Reset signal (asynchronous active low).
- Performs arithmetic operations (ADD, SUB) and logical operations (AND, OR, XOR, NOT, SLL, SRL) governed by a 3-bit opcode.
- Out value loaded on a positive clock edge according to the driven opcode.
- Zero flag active high whenever out value is 0.
- Carry flag active high whenever an arithmetic addition generates a 9th bit.
- Overflow flag active high whenever signed arithmetic results cross the MSB limit.

Signals:

1. **RST_N** → Asynchronous active low; all output signals must be zero.
2. **A, B (8-bit inputs)** → Valid or invalid transaction data.
3. **Opcode (3-bit input)** → At posedge CLK, instructs the datapath multiplexing.
4. **Out (8-bit output)** → Checker:
   - If reset asserted → check out == 0.
   - At posedge CLK, validate output against reference model:
   - If opcode == ADD → check out == $A + B$.
   - If opcode == SUB → check out == $A - B$.
   - If opcode is logical → check out matches corresponding logical operator behavior.
5. **Zero** → Checker: if out == 0 → check zero == 1, else == 0.
6. **Carry** → Checker: if ADD result exceeds 8 bits → check carry == 1, else == 0.
7. **Overflow** → Checker: if sign bit is flipped during arithmetic → check overflow == 1, else == 0.

Functional coverage:

We need to cover all operational signals:

- **Opcode** → cover all 8 distinct instructions.
- **Inputs A and B** → cover critical bit-patterns (e.g., 8'h00, 8'hFF).
- **Out** → cover walking ones and walking zeros.
- **Zero Flag** → 1 iff (out == 0).
- **Carry Flag** → Toggle coverage, crossed with ADD opcode.
- **Overflow Flag** → Toggle coverage, crossed with ADD and SUB opcodes.

Sequences & Constrained Randomization:

1. **Reset sequence** → with $RST\_N == 0$.
2. **Arithmetic sequence** → with opcode constrained to 3'b000 or 3'b001.
3. **Logical sequence** → with opcode constrained between 3'b010 and 3'b111.
4. **Corner-case sequence** → operands A and B constrained to max/min boundaries to stress flag evaluation.

Test scenarios:

- **Scenario 1:** Reset sequence → Arithmetic sequence for random number.
- **Goal:** Testing datapath addition/subtraction and the precise generation of carry/overflow flags.
- **Scenario 2:** Reset sequence → Logical sequence for random number.
- **Goal:** Testing masking, shifting, logical operations and zero-flag evaluations.
- **Scenario 3:** Reset sequence → Arithmetic sequence → Logical → Corner-case.
- **Goal:** Mixing the order of sequences to verify stability.
- **Scenario 4:** Fully unconstrained random sequence for 100 times.
- **Goal:** Testing overall system functionality and uncovering deep corner cases.